Materia: Programación IV

UNIDAD 7 · SERVER STATE Y NAVEGACIÓN

APUNTE TEÓRICO

React Router con TypeScript

Revisor de la unidad: Prof. Franco González

BrowserRouter · Routes · useNavigate · useParams

1. Introducción al enrutamiento en React

En las aplicaciones web tradicionales, cada vez que el usuario navega a una nueva página, el navegador solicita un nuevo documento HTML al servidor. Este proceso implica una recarga completa de la página.

Sin embargo, las aplicaciones modernas construidas con **React** suelen funcionar como **Single Page Applications (SPA)**.

Una **Single Page Application** es una aplicación que carga una sola página HTML inicial y luego actualiza el contenido dinámicamente mediante JavaScript. En lugar de recargar toda la página, React actualiza únicamente los componentes necesarios.

Esto permite:

- Navegación más rápida.
- Experiencia de usuario más fluida.
- Menor transferencia de datos en cada cambio de vista, ya que no se descarga un documento HTML completo.
- Conservación del estado de la aplicación al cambiar de vista.

React no incluye un sistema de rutas propio; para gestionar la navegación dentro de una SPA se utilizan librerías especializadas. Una de las más difundidas es **React Router**.

2. ¿Qué es React Router?

React Router es una librería que permite gestionar la navegación dentro de aplicaciones React.

Su objetivo principal es **asociar rutas de la URL con componentes específicos de la interfaz**.

Cuando la URL cambia, React Router decide qué componente debe renderizarse.

Por ejemplo:

URL	Componente
`/`	Home
`/productos`	Lista de productos
`/producto/1`	Detalle del producto

De esta forma, cada ruta representa una **vista diferente dentro de la aplicación**.

React Router permite:

- Definir rutas dentro de la aplicación.
- Renderizar componentes según la URL.
- Navegar sin recargar la página.
- Trabajar con rutas dinámicas.
- Extraer parámetros desde la URL.

Esto permite construir aplicaciones complejas con múltiples vistas manteniendo la experiencia fluida de una SPA.

3. Instalación de React Router

Para utilizar React Router en una aplicación React es necesario instalar la librería **react-router-dom**.

En proyectos creados con **Vite** y utilizando **pnpm**, la instalación se realiza con el siguiente comando:

```
pnpm add react-router-dom
```

En la versión 7 de React Router, el paquete principal es `react-router` y `react-router-dom` lo reexporta por compatibilidad, de modo que las importaciones desde `react-router-dom` que se muestran en este documento siguen siendo válidas.

Esta librería incluye:

- Componentes para definir rutas.
- Hooks para navegar dentro de la aplicación.
- Herramientas para trabajar con parámetros de URL.

Una vez instalada la librería, se puede configurar el sistema de rutas.

4. BrowserRouter: habilitación del sistema de rutas

El primer paso para utilizar React Router es envolver la aplicación con el componente **BrowserRouter**.

Este componente provee el **contexto de navegación** necesario para que todos los componentes puedan interactuar con el sistema de rutas.

BrowserRouter utiliza la **History API del navegador** para manejar cambios en la URL sin recargar la página. Gracias a esa API, las URL son limpias (`/productos`) en lugar de incluir un numeral (`/#/productos`). Como esas rutas no existen como archivos en el servidor, en producción el servidor debe responder con `index.html` para cualquier ruta de la aplicación; el servidor de desarrollo de Vite ya lo hace de forma automática.

Ejemplo:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

Se coloca en el archivo de entrada de la aplicación, que en los proyectos creados con Vite es `src/main.tsx`.

Esto garantiza que **todos los componentes dentro de la aplicación puedan acceder al sistema de navegación**.

5. Definición de rutas con Routes y Route

Una vez habilitado el router, el siguiente paso es definir las rutas de la aplicación.

Para esto React Router proporciona dos componentes:

Routes

Es el contenedor que agrupa todas las rutas de la aplicación.

Su función es evaluar la URL actual y decidir qué componente debe renderizarse.

Route

Define una ruta específica dentro de la aplicación.

Cada `Route` tiene dos propiedades principales:

- **path**: la URL que se desea asociar.
- **element**: el elemento JSX que se renderiza cuando la URL coincide con `path`, por ejemplo `<Home />`.

Ejemplo:

```
import { Routes, Route } from "react-router-dom";

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/productos" element={<Productos />} />
</Routes>
```

En este caso:

- **/** muestra el componente Home
- **/productos** muestra el componente Productos

Este bloque se ubica habitualmente en el componente `App`, que queda dentro del `BrowserRouter` definido en `main.tsx`.

6. Rutas estáticas

Una **ruta estática** es aquella cuyo path es fijo.

Ejemplo:

```
/productos
/contacto
/inicio
```

Estas rutas siempre renderizan el mismo componente.

Ejemplo en código:

```
<Route path="/productos" element={<Productos />} />
```

Las rutas estáticas se utilizan generalmente para:

- Páginas principales.
- Listados.
- Paneles de control (dashboards).
- Secciones informativas.

7. Rutas dinámicas

En muchas aplicaciones es necesario mostrar contenido específico según un identificador.

Por ejemplo:

- Detalle de un producto.
- Perfil de un usuario.
- Detalle de un pedido.

Para estos casos se utilizan **rutas dinámicas**.

React Router permite definir segmentos variables en la URL utilizando la sintaxis **:parametro**.

Ejemplo:

```
<Route path="/producto/:id" element={<DetalleProducto />} />
```

Aquí:

- **:id** representa un valor dinámico.
- Ese valor se captura desde la URL.

Ejemplo de URLs válidas:

```
/producto/1
/producto/5
/producto/42
```

Todas estas URL renderizan el mismo componente; lo único que cambia es el valor del parámetro `id`.

8. useNavigate: navegación programática

React Router ofrece el hook **useNavigate** para cambiar de ruta desde el código.

Este hook devuelve una función llamada **navigate** que permite redirigir al usuario a otra ruta.

Ejemplo:

```
import { useNavigate } from "react-router-dom";

function BotonVolver() {
  const navigate = useNavigate();
  return <button onClick={() => navigate("/productos")}>Volver a productos</button>;
}
```

Al ejecutarse `navigate`, cambia la URL y React Router renderiza el componente asociado a la nueva ruta. La llamada se realiza dentro de un manejador de eventos (por ejemplo, el clic de un botón) o de un efecto, nunca directamente durante el renderizado.

Ejemplo con parámetro dinámico

```
navigate(`/producto/${id}`);
```

Esto permite construir rutas dinámicas utilizando variables.

Casos comunes de uso:

- Botones.
- Formularios.
- Redirecciones después de guardar datos.
- Navegación basada en lógica.

La documentación oficial de React Router recomienda el componente `Link` para la navegación que inicia el usuario, por ejemplo `<Link to="/productos">Productos</Link>`, y `useNavigate` para la navegación programática que ocurre como resultado de una acción o de la lógica de la aplicación, como después de guardar un formulario. Los ejemplos con botones de este documento (`BotonVolver` y el botón «Ver detalle» de la sección 10) usan `navigate` para mostrar el funcionamiento del hook; cuando un clic solo lleva a otra vista, la opción recomendada es `Link`, por ejemplo `<Link to={`/producto/${producto.id}`}>Ver detalle</Link>`.

9. useParams: lectura de parámetros

Cuando se trabaja con rutas dinámicas, es necesario acceder a los valores que vienen en la URL.

Para esto se utiliza el hook **useParams**.

Ejemplo:

```
import { useParams } from "react-router-dom";

const { id } = useParams<{ id: string }>(); // id: string | undefined
```

La llamada se escribe dentro del componente de destino. El tipo indicado entre `< >` declara qué parámetros tiene la ruta, lo que habilita el autocompletado y permite detectar errores en el nombre del parámetro.

Si la URL es:

```
/producto/10
```

El valor de **id** es:

```
"10"
```

Los valores de los parámetros **se reciben como texto**. En TypeScript, el tipo de `id` es `string | undefined`, porque el hook no puede garantizar que el parámetro exista en la ruta actual; por eso, antes de usarlo, se comprueba que esté definido.

Como los parámetros llegan como texto, si se necesita trabajar con números se convierten con `Number`:

```
if (!id) return <p>Producto no encontrado</p>;

const numericId = Number(id);
if (Number.isNaN(numericId)) return <p>Identificador inválido</p>;
```

`Number` devuelve `NaN` cuando recibe `undefined` o un texto que no representa un número, por ejemplo `Number("abc")`; las dos comprobaciones evitan trabajar con un identificador inválido.

10. Flujo completo de navegación

Un flujo típico de navegación en React Router funciona de la siguiente manera:

1. El usuario visualiza una lista de elementos (por ejemplo productos).
2. El usuario hace clic en uno de ellos.
3. El manejador del clic ejecuta la función `navigate` que devuelve `useNavigate`.
4. La aplicación cambia la URL a una ruta dinámica.
5. React Router detecta la nueva URL.
6. Se renderiza el componente asociado a esa ruta.
7. El componente de destino utiliza `useParams` para leer el parámetro.

Ejemplo:

```
Usuario
↓
Lista de productos
↓
Clic en producto
↓
navigate("/producto/1")
↓
URL cambia
↓
React Router renderiza DetalleProducto
↓
useParams obtiene id
```

El siguiente ejemplo reúne el flujo completo en dos componentes. `Productos` define el tipo `Producto`, muestra una lista y navega al detalle con `useNavigate`:

```tsx
import { useNavigate } from "react-router-dom";

type Producto = {
  id: number;
  nombre: string;
};

const productos: Producto[] = [
  { id: 1, nombre: "Pizza" },
  { id: 2, nombre: "Hamburguesa" },
  { id: 3, nombre: "Empanadas" },
];

export default function Productos() {
  const navigate = useNavigate();

  const verProducto = (id: number) => {
    navigate(`/producto/${id}`);
  };

  return (
    <ul>
      {productos.map((producto) => (
        <li key={producto.id}>
          {producto.nombre}
          <button onClick={() => verProducto(producto.id)}>Ver detalle</button>
        </li>
      ))}
    </ul>
  );
}
```

`DetalleProducto` lee el identificador desde la URL con `useParams`:

```
import { useParams } from "react-router-dom";

export default function DetalleProducto() {
  const { id } = useParams<{ id: string }>();

  if (!id) return <p>Producto no encontrado</p>;

  return (
    <div>
      <h1>Detalle del producto</h1>
      <p>ID del producto: {id}</p>
    </div>
  );
}
```

Este mecanismo permite construir aplicaciones dinámicas donde la URL representa el estado de navegación. En una aplicación conectada a una API, el componente de detalle usa ese parámetro para pedir los datos del recurso; con TanStack Query, el identificador forma parte de la clave de la consulta (por ejemplo, `queryKey: ["producto", id]`), de modo que cada producto se almacena en caché por separado.

11. Conclusión

React Router asocia URL con componentes y permite navegar entre vistas sin recargar la página.

Permite estructurar la navegación de manera clara y organizada, separando la aplicación en diferentes vistas asociadas a rutas.

Los elementos trabajados en este documento son:

- **BrowserRouter**: habilita el sistema de rutas.
- **Routes**: contenedor de rutas.
- **Route**: define una ruta específica.
- **useNavigate**: permite navegar desde el código.
- **useParams**: permite leer parámetros dinámicos de la URL.

Con estos elementos, la URL indica qué vista se muestra y qué recurso se consulta, de modo que cada vista de la aplicación queda identificada por una dirección propia.

Bibliografía

1. Banks, A., & Porcello, E. (2020). *Learning React* (2.ª ed.). O'Reilly Media.
2. Microsoft. (s. f.). *TypeScript handbook*. https://www.typescriptlang.org/docs/handbook
3. React. (s. f.). *React documentation*. https://react.dev
4. React Router. (s. f.). *React Router documentation*. https://reactrouter.com